

C++ Programming: Systems And Programming Concepts

Chapter 4 Pointers

Objective

- Pointer variable
- Pointer addresses
- Pointer manipulations

Introduction

- Every variable has a memory location in the memory. Every memory location has an address which is accessed by an ampersand (&)

- Example:

```
int var1;
```

```
char var2;
```

```
cout << "Address of var1 : " << &var1;
```

```
cout << "Address of var2 : " << (void *)&var2;
```

Pointer/Pointer Variable

- A **pointer** is a variable whose **value** is the **address of another variable** (storing the address of the variable that the pointer is pointing to).

P (pointer P stores address only)

BC231

10FFA

variable A

10FFA

300

Pointer Declaration

Must have an asterisk before *each* of the pointer variables.

```
type *pointer variable name;
```

Example:

```
int *ip;      // pointer to an integer var.
```

```
double *dp;   // pointer to a double var.
```

```
char *ch;     // pointer to charact
```

Pointer variable can only point to the variable of the same type.

Pointer Declaration

The asterisk `*` can appear anywhere between the type and variable name

Example:

- `int *p;`
- `int* p;`
- `int * p;`
- `int*p;`
- `int *p, q;` (p is pointer variable, not q)
- `int *p, *q;` (both p and q are pointer variables)

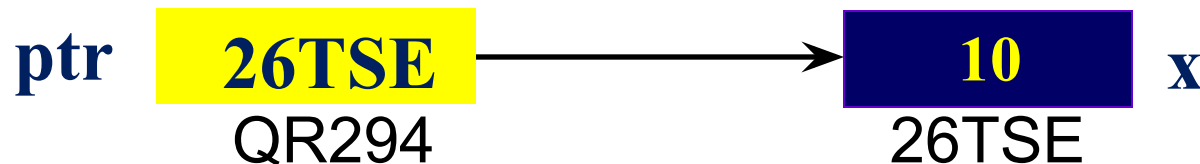
Assign Value to Pointer

The pointer declaration allocates memory space for the pointer variable, but **does not make it point to anything**. E.g. `int *ptr;`

To make a pointer point to something, the **address operator (&)** is used. This assigns address value to the pointer variable.

E.g.: `int *ptr, x = 10 ;`

`ptr = &x ;` //assign the address of x to ptr
// or pointer ptr points to x



Assign Value to Pointer

double *number; **not equal to** ***number**

↙
Declare a double
type pointer

↙
Reference to
variable

double a =10;

double *number = &a; // 1 line, is the same as

double *number;
number = &a; **] // 2 lines**

8 **** But not *number = &a; X**

Dereferencing Operator (*)

When used as a unary operator, *

Called **dereferencing** operator or **indirection** operator

Refers to object or variable to which its pointer is pointing to

Returns the value stored in the variable or stores a value into the variable through the pointer variable

E.g. `int num = 10, *ptr = #`

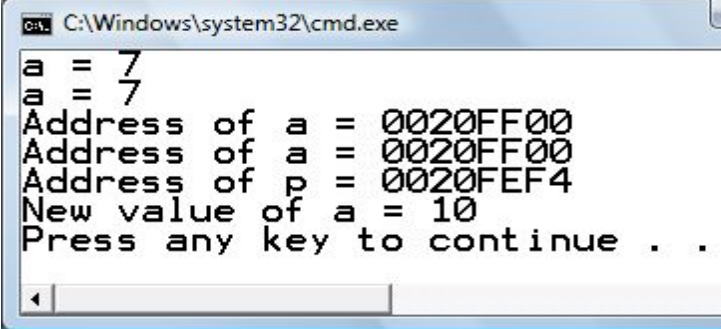
`cout << *ptr; // return 10 as output`

`*ptr = 20; // store 20 into num through *ptr`

Dereferencing Operator (*)

//Program illustrates the distinction between a pointer
//value and its dereferenced value

```
int main( ) {  
    int a=7, *p=&a;  
    cout << "a = " << a << endl;  
    cout << "a = " << *p << endl;  
    cout << "Address of a = " << &a << endl;  
    cout << "Address of a = " << p << endl;  
    cout << "Address of p = " << &p << endl;  
    *p=10;  
    cout<<"New value of a = " << a << endl;  
    return 0;  
}
```



The screenshot shows a Windows command prompt window titled "C:\Windows\system32\cmd.exe". The output of the program is as follows:

```
a = 7  
a = 7  
Address of a = 0020FF00  
Address of a = 0020FF00  
Address of p = 0020FEF4  
New value of a = 10  
Press any key to continue . . .
```

Initializing Pointer Variable

- C++ does not automatically initialize variables

- Pointer variables do not point to any valid object, it can be initialized using the **constant value 0**, called the **null pointer**

- The following two statements are equivalent:

```
p = NULL; // predefined constant  
p = 0;
```

Pointer - Example

```
#include <iostream>
using namespace std;
int main ( ) {
    int value1 = 5, value2 = 15;
    int *mypointer = NULL;
    mypointer = &value1;
    *mypointer = 10;
    mypointer = &value2;
    *mypointer = 20;
    cout<<value1<<"/"<<value2;
    return 0;
```

Output:
10/20

Pointer - Example

```
#include <iostream>
using namespace std;
int main ( ) {
    int value1 = 5, value2 = 15, *p1, *p2;
    p1 = &value1;
    p2 = &value2;
    *p1 = 10;
    *p2 = *p1; // assign value
    p1 = p2;   // assign address
    *p1 = 20;
    cout<<value1<<"/"<<value2<<endl;
    return 0; }
```

Output:
10/20

Dynamic Variables using **new**

- Variables can be manipulated even if the variables have no identifiers to name them.
- The operator **new** can be used to create variables that have no identifiers to serve as their names. These nameless variables are referred to *via pointers*.
- Example:

```
int *p;
```

```
p = new int;    // p points to nameless var.
```

Dynamic Variables using **new**

```
//Program to demonstrate pointers and dynamic variables.
#include <iostream>
using namespace std;

int main()
{
    int *p1, *p2;

    p1 = new int;
    *p1 = 42;
    p2 = p1;
    cout << "*p1 == " << *p1 << endl;
    cout << "*p2 == " << *p2 << endl;

    *p2 = 53;
    cout << "*p1 == " << *p1 << endl;
    cout << "*p2 == " << *p2 << endl;

    p1 = new int;
    *p1 = 88;
    cout << "*p1 == " << *p1 << endl;
    cout << "*p2 == " << *p2 << endl;
    return 0;
}
```

Output:

*p1 == 42

*p2 == 42

*p1 == 53

*p2 == 53

*p1 == 88

*p2 == 53

Pointer Arithmetic

You can perform arithmetic operations on a pointer just as you can do on a numeric value, such as using:

- Increment (++) operator

- Decrement (--) operator

- Assignment (=) operator

- Addition (+) operator

- Subtraction (-) operator

Pointer Arithmetic

Each time a pointer is incremented by 1, it points to the memory location of the next element of its base type.

If “p” is a character pointer then “p++” will increment “p” by 1 byte.

If “p” is an integer pointer then “p++” will increment “p” by 4 bytes.

Pointer Arithmetic

Suppose we have:

`char *mychar;` //address 1000

`short *myshort;` //address 2000

`long *mylong;` //address 3000

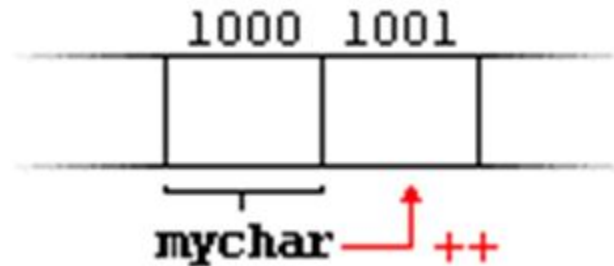
If we write: `++mychar;`

`++myshort;`

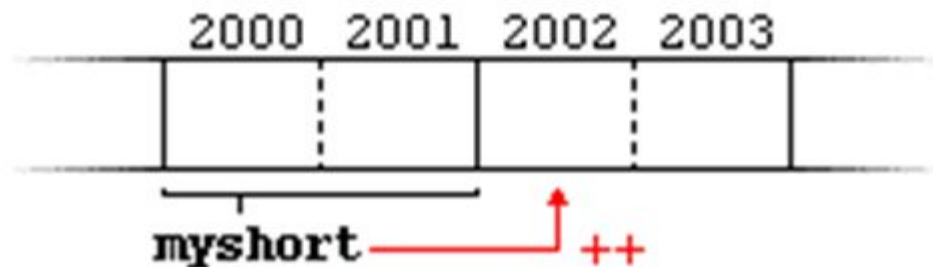
`++mylong;`

Pointer Arithmetic

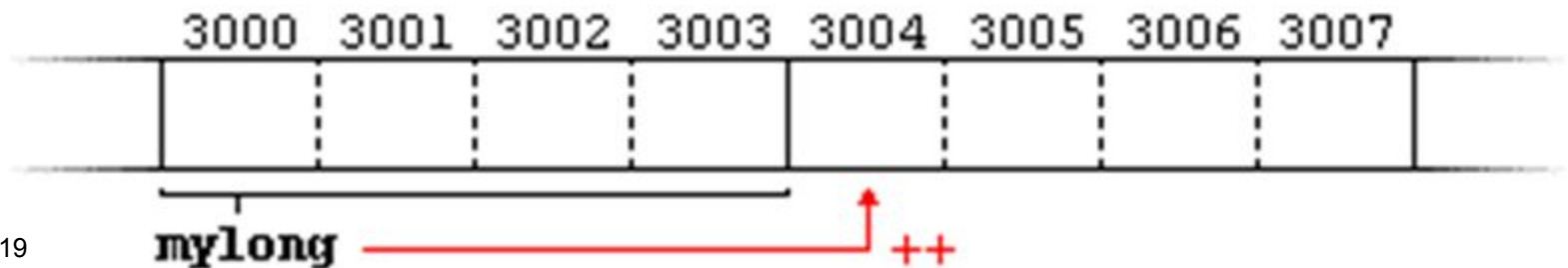
++mychar;



++myshort;



++mylong;



Pointer Arithmetic - Example

```
#include <iostream>
using namespace std;
int main() {
    int num = 7;
    int *pnPtr = &num;
    cout << pnPtr << endl;
    cout << pnPtr+1 << endl;
    cout << pnPtr+2 << endl;
    cout << pnPtr+3 << endl;
    return 0;
}
```

Pointer Arithmetic - Example

```
int main() {  
    int *x, *p, *q, c = 100, a;  
    x = &c;  
    p = x + 2;  
    q = x - 2;  
    a = p - q;  
    cout << "The address of x : " << x << endl;  
    cout << "The address of p after incrementing x by 2 : "  
<< p << endl;  
    cout << "The address of p after decrementing x by 2 : "  
<< q << endl;  
    cout << "The number of elements between p and q : "  
<<21 a << endl; return 0; }
```

The use of dereference pointer (*)

Four possible combinations:

***p++** // same as ***(p++)**

increment pointer, and dereference unincremented (postfix) address

***++p** // same as ***(++p)**

increment pointer, and dereference incremented (prefix) address

++*p // same as **++(*p)**

dereference pointer, and increment the value it points to

(*p)++

dereference pointer, and post-increment the value it points to

Reference Variable

A reference variable (or reference) is an *alias*, or an *alternate name* to an existing variable.

A reference is similar to a pointer.

To declare a *reference variable*, **&** is used in the *declaration*.

Syntax:

```
type &newName = existingName;
```

It shall be read as "*newName is a reference to existingName*", or "*newNew is an alias of existingName*".

Reference Variable - Example

```
#include <iostream>
using namespace std;
int main()
{   int number = 88;
    int & refNumber = number;
    cout << number << endl;
    cout << refNumber << endl;
    refNumber = 99;
    cout << refNumber << endl;
    cout << number << endl;
    number = 55;
    cout << number << endl;
    cout << refNumber << endl;
    return 0; }
```

Pointers - Exercise

- a) Declare a pointer variable *ptrNum* pointing to a double variable.
- b) Declare a double variable *num*, and set *ptrNum* to be pointing to the variable *num*.
- c) Assign the value 123 to the variable *num* through the pointer *ptrNum*.

Pointers - Exercise

Trace the output of the following program segment. Assume that the variable **a** is stored at the address 0x32AB and **b** at 0xE1F2.

```
int *ptr1, *ptr2, *ptr3, a = 10, b = 20;
ptr2 = ptr3 = &a;
ptr1 = &b;
cout << *ptr2 + b << endl;
cout << *&*ptr2 << endl ;
ptr3 = &b ;
a += *ptr3 + *ptr1 ;
ptr1 = ptr2;
cout << a << endl << *ptr3 + *ptr1 << endl;
cout << ptr1 << endl << ptr3 << endl;
```

Pointers - Exercise

Evaluate the following output:

```
int *aPtr, *bPtr;  
int a, b, c;  
aPtr = &a;  
bPtr = &b;  
*aPtr = 12;  
b = 13;  
cout << "a = " << a << endl;  
cout << "b = " << *bPtr << endl;  
c = a + *bPtr;  
cout << "c = " << c << endl;  
aPtr = &c;  
a = *aPtr - 2;  
(*bPtr)++;  
cout << "a = " << a << endl;  
cout << "b = " << b << endl;
```